

SQL Injection

PortSwigger Web Security Academy — Lab Notes

A consolidated, cleaned-up set of notes covering five hands-on labs: hidden data retrieval, login bypass, and UNION-based injection across Oracle, MySQL, and Microsoft SQL Server — including full database enumeration via `information_schema`.

Contents

1. Lab 1 — Retrieving Hidden Data (Category Filter)
2. Lab 2 — SQL Login Bypass
3. Lab 3 — UNION Injection: Oracle Database Version
4. Lab 4 — UNION Injection: MySQL / MS SQL Server Version
5. Lab 5 — Listing Database Contents (`information_schema`)
6. Consolidated Key Takeaways & Cheat Sheet

Retrieving Hidden Data — Category Filter

Original SQL Query

```
SELECT * FROM products
WHERE category = 'Gifts'
AND released = 1;
```

- **SELECT *** → return all columns.
- **FROM products** → read from the products table.
- **WHERE category='Gifts'** → only products in the Gifts category.
- **AND released=1** → only products that have been released — unreleased products (released=0) stay hidden.

Injection Point

The vulnerable parameter is category, e.g. ?category=Gifts. Only the value of *category* is inserted into the SQL query.

Important Discovery

Trying ?category=Gifts&released=0 does **not** work, because the application never reads a *released* URL parameter at all — it's hardcoded server-side as `AND released = 1`. Adding an extra HTTP parameter has no effect.

Core idea: the goal isn't to add a new URL parameter — it's to change the SQL query itself, by injecting SQL inside the existing value.

Anatomy of the Payload

' — closing the string

Closes the existing SQL string. Without it, everything you type stays part of the category name (plain text, not SQL).

OR 1=1 — always true

1=1 always evaluates to TRUE, so category='Gifts' OR 1=1 becomes true for every row in the table.

-- — comment

Starts a SQL comment; everything after it is ignored — this is what removes `AND released=1` from execution.

URL Encoding Note

A + in a URL parameter represents a **space**. So OR+1=1 decodes to OR 1=1.

SQL Login Bypass

Original Query

```
SELECT *  
FROM users  
WHERE username='administrator'  
AND password='secret';
```

Goal

Don't guess the password — remove the password check entirely, so the logic effectively becomes just WHERE username='administrator'.

Two Distinct Questions (clarified)

Which HTML field receives the payload? → the *Username* field.

Which SQL condition are we trying to eliminate? → AND password='... '.

Both answers are correct — they're just answering two different questions (injection point vs. logic being bypassed), not contradicting each other.

UNION Injection — Oracle Database Version

This lab introduces a new concept. Previous labs modified existing SQL logic; this one executes an **entirely separate SELECT statement** and merges its results with the original query's output.

What UNION Does

```
SELECT ...  
UNION  
SELECT ...
```

The database returns one combined result set from both SELECT statements.

Step 1 — Find the Number of Columns

- ORDER BY 1 → worked
- ORDER BY 2 → worked
- ORDER BY 3 → Internal Server Error

Conclusion: the original query returns exactly 2 columns. If the query returns Column1 and Column2, ordering by 1 or 2 works, but ordering by a nonexistent 3rd column fails — the first failure reveals the total column count.

UNION Rule

The injected SELECT must return the **exact same number of columns** as the original. Since this query has two columns, the injection must be UNION SELECT value1, value2 — a single value would fail.

Finding Which Column Displays Text

```
UNION SELECT 'abc', NULL  
UNION SELECT NULL, 'abc'
```

Purpose: discover *where* text gets displayed on the page. 'abc' is just a literal string marker — not a column, variable, or table — it simply asks the database to return the exact text "abc".

Oracle Quirk — FROM dual

Unlike MySQL/PostgreSQL, Oracle generally does not allow SELECT 'abc' without a table — it requires FROM dual:

```
SELECT 'abc'  
FROM dual;
```

DUAL is Oracle's built-in dummy table, used purely to return literal values without reading real data.

On Evidence vs. Assumption

Initially, UNION SELECT 'abc', NULL FROM dual didn't visibly show "abc" on the page — but that alone wasn't enough evidence to conclude Oracle hadn't returned the row. In a later lab, repeating the same payload showed "abc" appearing at the bottom of the product list — most likely the earlier lab behaved the same way and it just wasn't scrolled into view. Lesson: never claim something happened without evidence, and update conclusions

when new evidence appears.

Retrieving the Oracle Version

Oracle stores version info inside `v$version`, in a column called `BANNER`:

```
SELECT BANNER
FROM v$version;
```

Returns rows such as: Oracle Database..., PL/SQL Release..., CORE..., TNS..., NLSRTL...

Full injected query:

```
UNION
SELECT BANNER, NULL
FROM v$version
```

This combines the original product rows with the Oracle version rows into one result set.

'abc' vs. BANNER

- **'abc'** is a literal string — Oracle just echoes it back.
- **BANNER** is a real column name — Oracle reads the actual stored values.

Biggest Conceptual Shift

Earlier labs manipulated the *existing* query (e.g. `OR 1=1` changing WHERE-clause logic). UNION-based labs instead run an entirely new `SELECT` and merge its answer into the original results.

Complete UNION Workflow

- Confirm SQL injection is present.
- Determine the number of columns (ORDER BY).
- Match that number of columns in the UNION SELECT.
- Determine which columns can display text ('abc' markers).
- Replace the text marker with useful data (e.g. database version).

UNION Injection — MySQL / MS SQL Server Version

Step 1 — Number of Columns

```
' ORDER BY 1--+
' ORDER BY 2--+
' ORDER BY 3--+
```

Results: ORDER BY 1 ■ ORDER BY 2 ■ ORDER BY 3 ■ → the query returns exactly 2 columns.

Why -- Failed but --+ Worked

In MySQL and Microsoft SQL Server, -- only starts a comment if it's followed by whitespace. The URL sequence --+ URL-decodes to -- followed by a space — so the database correctly recognizes it as a comment.

Step 2 — Which Columns Accept Text

```
UNION SELECT 'abc', NULL
UNION SELECT NULL, 'abc'
```

No FROM dual needed here. Both columns displayed "abc" directly — unlike the Oracle lab, this time there was direct visual evidence on the page.

Why No FROM dual?

Oracle requires every SELECT to read from a table, hence FROM dual. MySQL and MS SQL Server allow a bare SELECT 'abc'; with no dummy table needed — FROM dual is Oracle-specific.

Retrieving the Version

```
SELECT @@version;
```

@@version is not a table or column — it's a built-in **system variable** holding the current database version, e.g. 8.0.42-Ubuntu0.20.04.1.

Version Retrieval Differs by DBMS

Database	Version Query
Oracle	SELECT BANNER FROM v\$version;
MySQL	SELECT @@version;
Microsoft SQL Server	SELECT @@version;
PostgreSQL	SELECT version();

BANNER vs. @@version

- **Oracle:** BANNER is a column inside a system view — requires `SELECT BANNER FROM v$version`.
- **MySQL / MS SQL:** @@version is a built-in system variable — works standalone, no table needed.

Evidence vs. Assumption

The returned string "8.0.42-Ubuntu0.20.04.1" strongly matches MySQL's version format — that's the evidence. Concluding "this is MySQL" is the inference built on top of it. Always separate the two.

Listing Database Contents

Objective

Retrieve usernames and passwords from the application's database.

Standard Manual SQLi Workflow

- Confirm SQL injection.
- Find number of columns.
- Find text-display columns.
- Discover table names.
- Discover column names.
- Extract data.
- Use credentials.

What is information_schema?

A special database catalog containing **metadata** — information about the database itself (table names, column names, data types, schemas, views, permissions), not application data.

```
information_schema
■
■■■ tables
■■■ columns
■■■ views
■■■ routines
■■■ ...
```

information_schema.tables

Lists the names of every table:

```
SELECT table_name
FROM information_schema.tables;
```

Without a WHERE clause, this returns *every* table — application tables, internal tables, system tables, and framework tables alike, which is why the result set was so large.

Filtering the Results

```
WHERE table_name LIKE '%user%'
```

This quickly revealed the randomized table `users_vrefio`. Note: `pg_user` was intentionally skipped — it's a PostgreSQL *system* table/view, not the application's own login table.

information_schema.columns

Lists every column inside every table:


```
SELECT column_name
FROM information_schema.columns
WHERE table_name='users_vrefio';
```

A typo in the table name initially returned nothing; after correcting it, the query returned: email, password_bdbion, username_miefjv.

PortSwigger randomizes table/column names on purpose (e.g. users_vrefio instead of users) — this forces enumeration instead of guessing.

Dumping the Credentials

```
SELECT username_miefjv,
password_bdbion
FROM users_vrefio;
```

Result: all usernames and passwords, including administrator / l7yugzx0c52i9q5skpqs — used to log in as administrator. Lab solved.

Why Enumerate Instead of Guess?

Rather than guessing table names like *users*, *accounts*, *members*, or *customers*, we ask the database directly: what tables do you have → what columns does this table contain → give me the data. Enumeration via `information_schema` is far more reliable than guessing.

Parameter vs. Value — Clarifying "Gifts"

Original URL: `category=Gifts` → parameter = `category`, value = `Gifts`. We never invented "Gifts" — we modified the existing parameter's *value*, e.g.:

```
category=Gifts'+ORDER+BY+1--+
```

If another valid category existed (e.g. `Pets`), the same injection pattern applied to that value would typically work too. The vulnerability lives in the value being inserted directly into SQL, not in the parameter name.

Why Every Payload Starts With '

If the app executes `WHERE category='Gifts'` and we simply appended `ORDER BY 1` without closing the quote, it would become `WHERE category='Gifts ORDER BY 1'` — treated as plain text. Closing the string first with `'` lets everything afterward be interpreted as SQL code, and the trailing `--` comments out the rest of the original query.

Commas vs. +

- `,` separates SQL columns — e.g. `SELECT NULL,'abc'` (without it, the SQL is invalid).
- `+` is not SQL syntax at all — it only represents a space once the URL is decoded.

Consolidated Key Takeaways & Cheat Sheet

Core Principles

- SQL injection happens inside **existing parameter values** — not by adding new HTTP parameters.
- ' closes the existing SQL string, letting your injected code run as SQL rather than text.
- OR 1=1 changes query logic to always evaluate true.
- -- comments out the rest of the original query (needs trailing whitespace in MySQL/MS SQL — hence --+ in URLs).
- + in a URL represents a space after decoding — it is not SQL syntax.
- , separates columns in a SELECT/UNION statement.

UNION-Based Injection

- UNION combines two SELECT result sets into one.
- Both SELECTs must return the **same number of columns**.
- Use ORDER BY n to discover the column count — the first number that errors reveals the total.
- Use literal markers like 'abc' to discover which columns display as text.

Database-Specific Differences

	Oracle	MySQL / MS SQL Server	PostgreSQL
Literal SELECT	Needs FROM dual	Works standalone	Works standalone
Version query	SELECT BANNER FROM v\$version	SELECT @@version	SELECT version()

Database Enumeration (information_schema)

- information_schema.tables → lists every table name.
- information_schema.columns (filtered by table_name) → lists every column in a table.
- Filter with LIKE '%keyword%' to cut through system/framework noise.
- Enumerate — don't guess — table and column names, especially when they're randomized.

Scientific Mindset

Distinguish carefully between what evidence actually proves and what is merely inferred. Don't assume something happened without visible proof — and when new evidence turns up later, update your conclusion rather than clinging to the earlier assumption.